

TracePilot: Self-verifiable Framework for Decentralized Applications Fault Localization across Transactions

ANONYMOUS AUTHOR(S)

Decentralized Applications (DApps) serve as a critical technical underpinning for business logic and user interaction within the blockchain-powered Web3 ecosystem. However, DApps are prone to faults, and localizing these faults within their intricate and often interconnected logic is a particularly time-consuming process, frequently taking tens of hours and leading to substantial economic losses for developers. Existing state-of-the-art DApp fault localization methods, e.g., FaultSeeker, cannot capture cross-transaction fault logic and produce verifiable diagnostic reports. Therefore, the security experts have to spend substantial time manually verifying results and devising fixes. In this paper, we present TracePilot, an large language models (LLMs)-based framework that automates DApp fault localization in two phases: distilling global fault insights from transaction sequences and then performing focused trace exploration to isolate the faulty logic. Crucially, we propose a novel patch verification technique, which allows for verifying the correctness of DApp fault localization reports, rather than relying on expert verification. Evaluated on a dataset of 149 real-world cases, TracePilot achieves a 71.14% Top-1 Recall, 128.9% higher than the state-of-the-art methods. Moreover, to facilitate further research, our code and dataset are publicly available online: <https://anonymous.4open.science/r/TracePilot-F8A6/>.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; **Fault tree analysis**.

Additional Key Words and Phrases: fault localization, decentralized applications, Web3

ACM Reference Format:

Anonymous Author(s). 2018. TracePilot: Self-verifiable Framework for Decentralized Applications Fault Localization across Transactions. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 20 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

As a critical component of Web3 architecture, Decentralized Applications (DApps) are accelerating their evolution toward greater scale and complexity. As of January 2026, assets locked within DApps have reached \$129.31 billion [11]. This substantial concentration of assets has attracted increasingly sophisticated DApp attacks. Following an attack, post-incident analysis is of paramount importance for emergency vulnerability remediation, preventing the spread of exploits, and maintaining market stability [25, 35]. Unfortunately, the post-incident analysis of DApp faults is extremely laborious and relies heavily on expert experience. Previous research [35] indicates that 79.8% of the time in current post-incident analysis processes is consumed by fault localization, with human experts requiring an average of 16.7 hours to manually localize a DApp fault.

To alleviate the analysis burden on DApps security experts, existing research [14, 25, 34, 35, 37, 38] has attempted to introduce automated fault localization tools. However, current solutions often fall short when confronting complex cross-transaction DApp faults. Specifically, general techniques rely on comprehensive test suites or detailed error messages, whereas hackers often bypass exception

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

handling during DApp attacks to steal funds. Meanwhile, the technologies specifically designed for DApp faults also have some limitations. For instance, DAppFL [35] fails to comprehend the underlying fault logic and struggles to provide interpretable evidence. FaultSeeker [25] pioneers attention-driven LLM analysis for transaction faults but remains limited to single-transaction scope. This precludes the reconstruction of multi-step attack logic, and its reliance on rule-based filtering and lack of verification hinder its robustness and reliability. Moreover, all the existing DApp fault post-incident analysis methods omit a self-verification procedure. Without a validation, the accumulated hallucinations may compromise post-incident analysis and mislead human experts.

To build up a reliable DApp fault localization tool, there are three critical challenges:

- (1) **Complexity of cross-transaction DApp faults.** Specifically, state changes in preceding transactions serve as necessary prerequisites for subsequent transaction to trigger faults. Therefore, the state dependencies across multiple transactions exponentially increase both the complexity of analysis and the search space.
- (2) **Noise inherent in execution traces.** Complex transactions generate massive function calls. And the majority of functions are unrelated to the DApp fault [36]. Therefore, it is difficult to isolate the complete fault logic.
- (3) **Prohibitive Cost of Manual Verification.** Existing tools predominantly generate a list of suspicious functions. The unverified diagnostic results compel experts to invest additional time in assessing reliability. Moreover, false positives can mislead expert judgment and ultimately prolong the localization process.

In this paper, we propose TracePilot, an LLM-driven end-to-end framework for localizing faults in DApps. The core insight behind our approach is inspired by the systematic methodology employed by human experts in debugging DApps [9]. Through a structured analysis of expert fault reports, we identify a recurring cognitive pattern that guides efficient fault localization. This pattern consists of three distinct phases: hypothesis, analysis, and verification. Rather than exhaustively traversing execution traces, experts first formulate a high-level hypothesis based on macro-level anomalies during transaction sequences. They then drill down into micro-level execution traces to gather supporting evidence. Finally, they validate their conclusions through targeted trial-and-error, such as simulating patches or replaying transactions. TracePilot is designed to internalize this structured human reasoning cycle into an automated, LLM-powered workflow.

Drawing on the experience of human experts, TracePilot continuously try to revise the fault conclusions until it arrives at verifiable patch code. Firstly, TracePilot formulates hypotheses based on high-level fault manifestations, decomposes these hypotheses into concrete, executable analysis goals, and investigates the fault execution traces to uncover critical fault details. Guided by the evolving fault understanding and hypotheses, the LLM agent autonomously determines whether to deeply explore a specific execution branch or prune it from the search space. It allows the agent to bypass irrelevant logic and dynamically refine its analysis scope. Secondly, TracePilot introduces automated patch verification based on the continuously revised fault understanding. And we can validate the correctness of fault localization results, if the generated patches blocking the fault transaction in simulation environment. We performed a comprehensive evaluation on a dataset comprising 149 real-world DApp fault cases. These cases encompass a wide range of complex fault patterns and cross-transaction scenarios. Experimental results demonstrate that TracePilot significantly outperforms existing state-of-the-art tools in terms of both localization accuracy and verifiability.

The main contributions of this paper are as follows:

- **Cross-transaction fault localization:** We introduce an agent-based approach for cross-transaction trace analysis. To the best of our knowledge, this is the *first* end-to-end automated framework for DApp fault localization across transactions.
- **Patch verification:** We propose a verification method for DApp fault localization based on automated smart contract patching, providing objective and verifiable evidence for the correctness of fault localization results.
- **Benchmark and experiments:** We extend the current DApp fault dataset to 149 real-world cases by recovering the complete transaction sequences from existing records and incorporating additional attacks. Experimental results demonstrate that TracePilot accurately localizes the root cause in 71.14% of cases, outperforming state-of-the-art methods by 128.9%.

2 Background

2.1 Smart Contracts and State Persistence

Ethereum and EVM-compatible blockchains operate as transaction-based state machines [33]. The execution of a transaction can trigger complex state changes across multiple contracts. Crucially, the Ethereum state (Storage) is persistent; changes made by one transaction (e.g., initializing a liquidity pool) are preserved and serve as the precondition for subsequent transactions. This cross-transaction state dependency implies that analyzing an attack requires looking at the history of previous transactions, not just the exploit transaction itself.

2.2 Execution Traces and Forensic Analysis

Effective post-incident analysis of DApp faults requires granular examination of transaction behaviors. Consequently, analysts typically rely on platforms like Phalcon [5] and Tenderly [28] to reconstruct raw data into hierarchical execution traces for scrutiny. As shown in Fig. 1, an execution trace is a hierarchical tree structure that exhaustively records every operation, capturing not only external calls (interactions between contracts) but also internal calls (function invocations within a contract). Each operation serves as a trace node in the tree, including external calls, internal calls, log nodes, and state access nodes. Visibility into internal calls and storage accesses (e.g., SSTORE/SLOAD) is vital for fault localization. These operations reveal the precise logic branches and variable mutations that trigger vulnerabilities. However, our statistics indicate that complete execution trace trees contain an average of 3972 nodes, which is about 12 times the size of external call traces. Peak sizes can reach 130,000 nodes. This sheer volume significantly increases the data analysis pressure and difficulty.

2.3 Transaction Simulation and Data Collection

High quality analysis requires correct and detailed transaction data. The transaction simulation API from Tenderly [30] meets this requirement. Benefiting from the robust data parsing capabilities of Tenderly, the returned data includes well-parsed raw values, such as function signatures and storage slot values. After simulations, this API provides transaction data in JSON format as follows.

Simulation Information. The simulation field in the API response stores the execution status of the transaction replay. When a transaction fails and reverts during the replay, this field contains the revert reason and the error stack trace.

Transaction-related Data. The transaction_info field in the API response contains complete basic information, including the block number, sender address, recipient address, and so on. Furthermore, the API returns parsed data collected during simulation, such as logs, asset_changes,

and `balance_diff`. These data reflect detailed transaction-level changes and serve as the core data source for macro analysis.

Contract Data. The `contracts` field in the response includes information for all verified contracts involved in the transaction, such as contract source code, bytecode, and compilation artifacts like ABIs. This contract information serves as a solid foundation for toolkit construction. To support the processing of unverified contracts, we supplement missing runtime bytecode information by accessing the `getCode` API from JSON-RPC nodes.

Execution Trace Data. The `call_trace` field in the response provides detailed execution trace information in a tree structure. The simulation API provides a complete execution trace which is consistent with the Tenderly dashboard [28]. Each trace node contains detailed data such as caller information, callee information, call opcodes, and a list of sub-calls. Moreover, each node has a unique index that can be used to retrieve detailed data. To facilitate rapid access, we aggregate child state accesses and log nodes as direct attributes of each call node within the trace tree.

2.4 LLM-based Agents and Context Constraints

Large Language Models (LLMs) have demonstrated remarkable capabilities in code understanding and automated debugging. By leveraging techniques like Chain-of-Thought [32] prompting, LLMs can reason about vulnerability patterns. Furthermore, Multi-Agent Systems enhance this capability by assigning distinct roles to LLMs to collaboratively decompose and complete tasks. This approach has demonstrated effectiveness in fault localization tasks [23, 39, 40]. Despite these advancements, LLMs face inherent limitations:

Context Window. The token limit restricts the direct ingestion of massive execution traces.

Hallucination. While useful, LLMs are prone to generating plausible but incorrect reasoning. Notably, recent studies [43] warn of “hallucination amplification” in multi-agent systems, where non-factual information is propagated across the agent network. To mitigate this risk, incorporating a rigorous verification mechanism is indispensable for validating diagnostic results.

3 Motivating Example

To clearly characterize cross-transaction DApp faults and illustrate the limitations of existing methods, we employ the SushiSwap exploit as a motivating example. This incident occurred in January 2021 and resulted in a loss of approximately \$100,000.

Incident Description. Fig. 2 illustrates the process of the cross-transaction attack targeting SushiSwap. Specifically, the SushiMaker contract is a critical component of the SushiSwap DApp, responsible for converting transaction fees accumulated during trading into WETH. The `bridgeFor` function within SushiMaker is configured with reliable token exchange paths provided by the project team. These paths ensure price stability, as they are underpinned by high-liquidity trading pairs. However, as the DApp evolved, SushiSwap introduced a new token named DIGG. The root cause of the fault lies in the fact that the developers neglected to configure a reliable swap path for this newly added token within the `bridgeFor` function. In the absence of a configured path, `bridgeFor` returns a default DIGG-WETH path, attempting to convert DIGG directly into WETH. Exploiting this configuration oversight, the attacker established a DIGG-WETH trading pair with extremely low liquidity in the first transaction, which was manipulated by the attacker. Subsequently, in the second transaction, the attacker invoked the `convert` function of SushiMaker to start the fee conversion process. Consequently, the SushiMaker contract fell into the trap constructed by the attacker, exchanging high-value DIGG fees for a negligible amount of WETH, resulting in severe pecuniary damage to SushiSwap.

Why Existing Tools Fail. Existing approaches struggle to uncover the root cause of this incident due to their inherent limitations in scope and reasoning. By leveraging fund flow features,

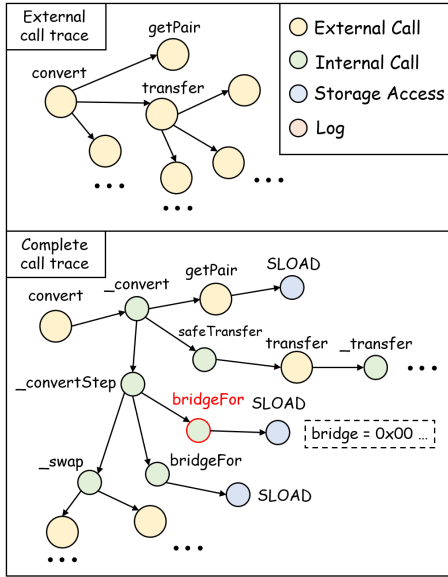


Fig. 1. External vs. Complete Call Traces.

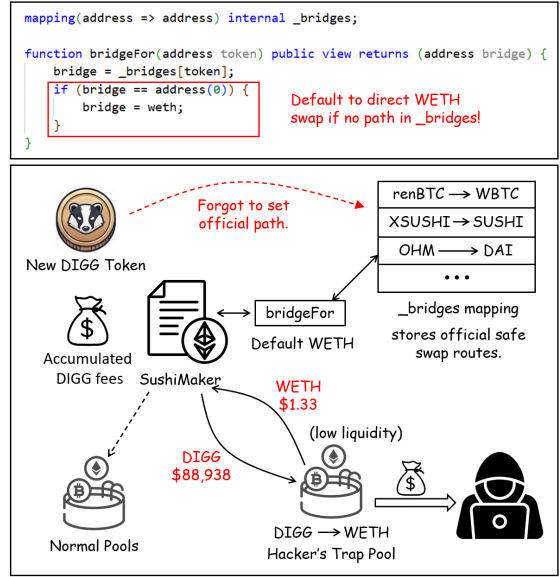


Fig. 2. Root Cause of the SushiSwap Exploit.

DAppFL [35] identifies swap functions as anomalies. The learning-based model lacks an understanding of code semantics, making it difficult to handle logic-related faults. As the state-of-the-art method, FaultSeeker [25] analyzes a single transaction and identifies `convert` as the most suspicious function instead of the root cause, `bridgeFor`. Since `convert` serves merely as the entry point, this result provides limited actionable insight. This misdiagnosis potentially occurs because `bridgeFor` does not exhibit typical anomalous characteristics, such as repeated invocations or abnormal parameters. Consequently, the heuristic rules in FaultSeeker filter out this critical function call. Additionally, FaultSeeker primarily bases its analysis on external call traces, as illustrated in Fig. 1. This focus limits its effectiveness in detecting root causes nested within internal calls, such as `bridgeFor`. Most fundamentally, without analyzing state dependencies across transactions, FaultSeeker cannot comprehend the attack intent, resulting in a severely limited fault understanding. Failing to recognize the DIGG-WETH pair as a pre-orchestrated trap, FaultSeeker misidentifies this maliciously created low-liquidity swap path as standard market behavior.

Our Approach. Unlike existing tools, TracePilot identifies root causes by combining involving fault understanding with dynamic analysis. Specifically, TracePilot identifies the creation of the low-liquidity DIGG-WETH pair in the first transaction as a critical precursor. It hypothesizes this creation is a necessary precondition rather than rational market behavior. Based on this, TracePilot dynamically explores the execution trace of the core attack transaction. It adaptively prunes benign calls to focus on fault-triggering segments. Finally, TracePilot suspects that the `bridgeFor` function contains suspicious logic. Subsequently, TracePilot conducts a fine-grained analysis of the `bridgeFor` function. By analyzing runtime state accesses alongside function source code, TracePilot pinpoints the specific logic flaw: When the function input is DIGG, the `_bridge` mapping returns a zero address, triggering the default situation. The `bridgeFor` returns WETH, setting the swap path to the malicious trading pair manipulated by attacker. Finally, TracePilot generates patches to validate the diagnosis. It injects checks into the `bridgeFor` function. These checks prevent transactions when the liquidity of trading pair is critically low. TracePilot validates the patches by replaying the attack process in a simulation environment. The successful interception confirms that the missing

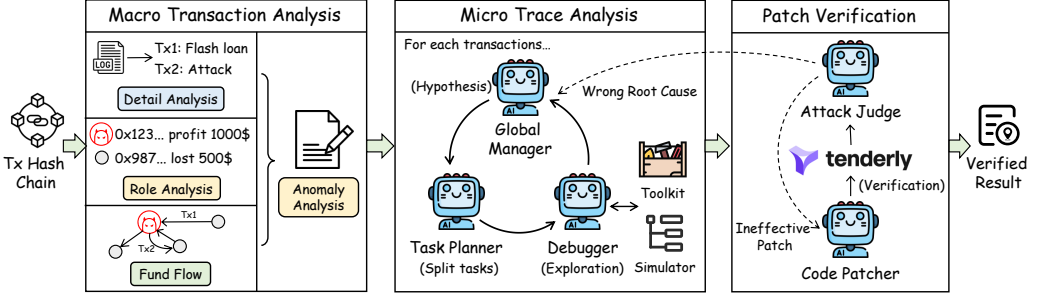


Fig. 3. Overall Workflow of the TracePilot Framework.

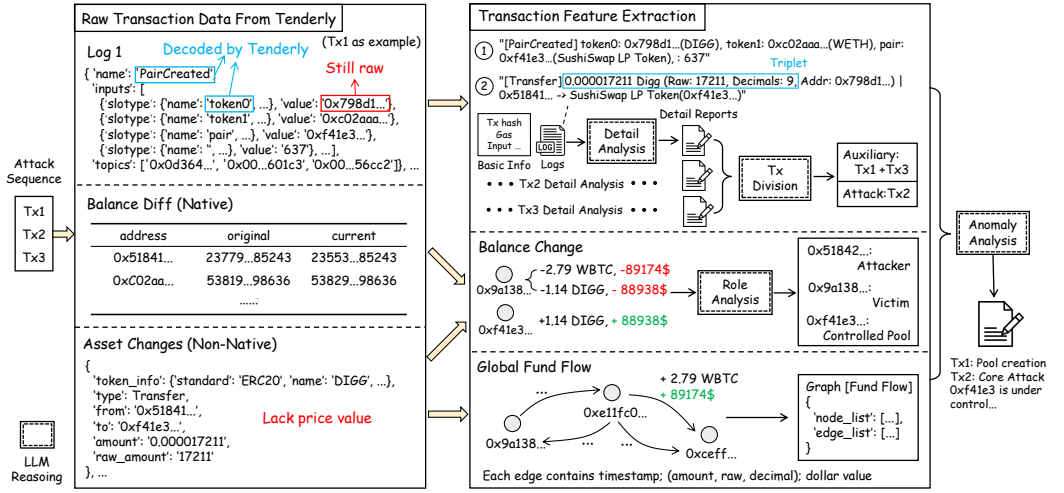


Fig. 4. Stage 1: Macro Transaction Analysis and Feature Extraction.

path configuration in bridgeFor is the root cause. This demonstrates that TracePilot effectively diagnoses the SushiSwap exploit and overcomes the limitations of existing tools.

4 Approach

4.1 Overview

As illustrated in Fig. 3, TracePilot operates in three stages: macro transaction analysis, micro trace analysis, and automated patch verification. Our work employs an iterative loop where macro hypotheses guide micro analysis, validated by a patch-based verification. TracePilot implements with a multi-agent architecture. The agents follow a workflow to analyze the DApp faults.

4.2 Macro Transaction Analysis

The workflow of macro transaction analysis is illustrated in Fig. 4. At this stage, we first utilize the `eth_getTransactionByHash` method from JSON-RPC nodes to obtain basic information for all transactions. Following this, our approach collects comprehensive transaction data from the transaction simulation API from Tenderly (detail in Section 2.3). This API provides a data source consistent with that used in expert analysis. Subsequently, transaction anomaly features are extracted by LLMs. Finally, these features are integrated to perform transaction anomaly analysis and obtain a macro-level understanding of transaction faults.

4.2.1 Transaction Feature Extraction. Although the simulation API provide partially parsed transaction data, the output remains highly complex, making direct analysis difficult. In practice, security experts rely on transaction analysis platforms to abstract the raw data into visual graphs. The graph representations allow experts to efficiently identify anomalous behaviors within transactions. Therefore, we extract transaction features through the following three steps.

Transaction Detail Analysis. As shown in Fig. 4, the detail analysis focuses on basic information and logs. Complex DApp faults often generate hundreds of logs cluttered with unscaled raw values. To enhance information density, TracePilot performs semantic enhancement and numerical normalization on transaction data. Specifically, we traverse logs and asset changes to collect a token information. Then, we resolve token contract addresses into readable symbols. Note that LLMs may not understand the raw token value in the transaction data, due to the decimals are different among tokens. Therefore, we standardize all value-related data into a numerical triplet, i.e., amount, raw value, and decimals. Next, we employ an LLM to summarize the behavior of each transaction from these details. After analyzing all transactions, we prompt the LLM to partition the attack sequence into auxiliary transactions and core attack transactions.

Transaction Role Analysis. Transaction role analysis aims to identify the functional identities of involved addresses across transactions. Our method evaluates the profit and loss for all involved addresses by tracking the balance changes in each transaction. TracePilot reconstructs the balance changes by synthesizing `balance_diff` (native token changes) and `asset_changes` (non-native token changes). To unify the measurement of asset values, we retrieve the historical token prices from the DeFiLlama API [10]. Building on the balance changes shown in Fig. 4, the LLM identifies the specific roles of involved addresses, such as the attacker, victim, or manipulated liquidity pools. The prompt is as follows:

Prompt: You are a Senior On-chain Transaction Fault Analyst. Given the balance changes, your objective is to assess potential victims and attackers in the attack, assign roles to each address involved in the transactions, and determine their function in the attack. {{Output Format}}

Fund Flow Graph Construction. As shown in Fig. 4, TracePilot constructs a cross-transaction global fund flow graph. By aggregating the asset change records from all transactions, the framework abstracts addresses as nodes and defines fund transfers as directed edges. Based on the fund flow graph, our method can delineate the complete attack trajectory from initial capital preparation to the final profit exfiltration. Given the proficiency of LLM in processing structured text, we employ an LLM-friendly semantic textual graph representation [31].

4.2.2 Transaction Anomaly Analysis. Building on the extracted transaction features, TracePilot synthesizes these semantic insights using LLM to generate a transaction anomaly report, establishing a qualitative understanding of the entire exploit. This analysis shows the attack logic by correlating state dependencies and fund flows across transactions. Specifically, our work links substantial asset anomalies to their underlying logical causes. In this way, it identifies key abnormal patterns, such as maliciously created low-liquidity pools and unexpected price slippage. This macro-level understanding serves as a definitive compass for the ensuing micro trace analysis.

4.3 Micro Trace Analysis

Following macro transaction analysis, we use a multi-agent architecture for micro trace analysis (Fig. 5). Existing research [25] shows that single agents often struggle to focus when facing complex attacks. This challenge is acute in cross-transaction attack scenarios. To address this challenge, we mimic the cognitive process of human experts by decomposing the analysis into manageable

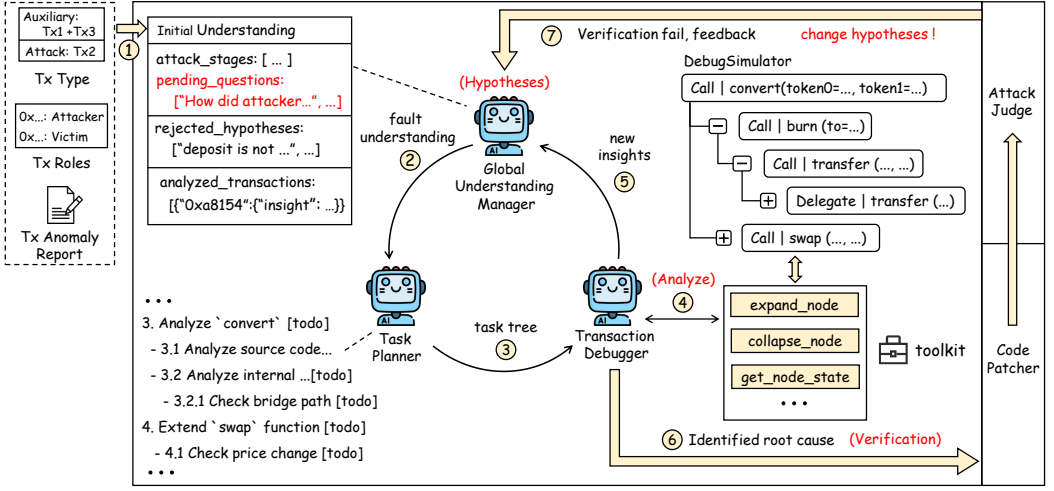


Fig. 5. Stage 2: Multi-agent Collaborative Micro Trace Analysis.

sub-tasks assigned to three specialized agents. This divide-and-conquer strategy ensures each task remains within the capabilities of agent while avoiding the overhead of excessive fragmentation.

4.3.1 Iterative Workflow. Our approach employs an iterative workflow with three agents: the global understanding manager, the task planner, and the transaction debugger. They convert high-level hypotheses into concrete analysis goals and then distill semantic insights from low-level data to improve fault understanding.

Global Understanding Manager. The global understanding manager serves as the high-level reasoning hub and hypothesis initiator. Mirroring human experts who prioritize querying specific anomalies over aimless code inspection, the manager maintains a dynamic pending_questions list. These questions act as unverified hypotheses, establishing semantic boundaries that prioritize suspicious trace segments and rapidly converge the search space. As analysis progresses, new insights are integrated into the fault understanding, while resolved questions generate subsequent hypotheses, forming an evolving reasoning loop. The global understanding manager employs a structured model (Fig. 5) to maintain the fault understanding, dynamically integrating new insights into this model. This incremental design ensures the attack logic remains unbroken and continues to evolve. For cross-transaction attacks, the global understanding manager maintains context by archiving findings into the analyzed_transactions field when switching targets. This ensures logical coherence across the entire attack sequence. Finally, the global understanding manager directs the workflow based on patch verification. Verified hypotheses form the final report, while failed ones are moved to rejected_hypotheses trigger a backtrack to explore other fault points.

Task Planner. Inspired by hierarchical task decomposition in LLM-based reasoning [15, 25, 27], the task planner dynamically maintains an analysis task tree (Fig. 5). It decomposes abstract hypotheses into fine-grained tasks to guide the subsequent analysis of transaction debugger.

Transaction Debugger. The transaction debugger performs analysis following the tasks. Acting as a security auditor, it uses a toolkit based on Model Context Protocol (MCP) [3] to explore and analyze execution traces. The agent selects tools dynamically based on its analysis results. This approach ensures analytical precision and maintains the agent focus on the core exploit logic, as detailed in Section 4.3.2. To improve efficiency, the agent can call multiple tools in a single

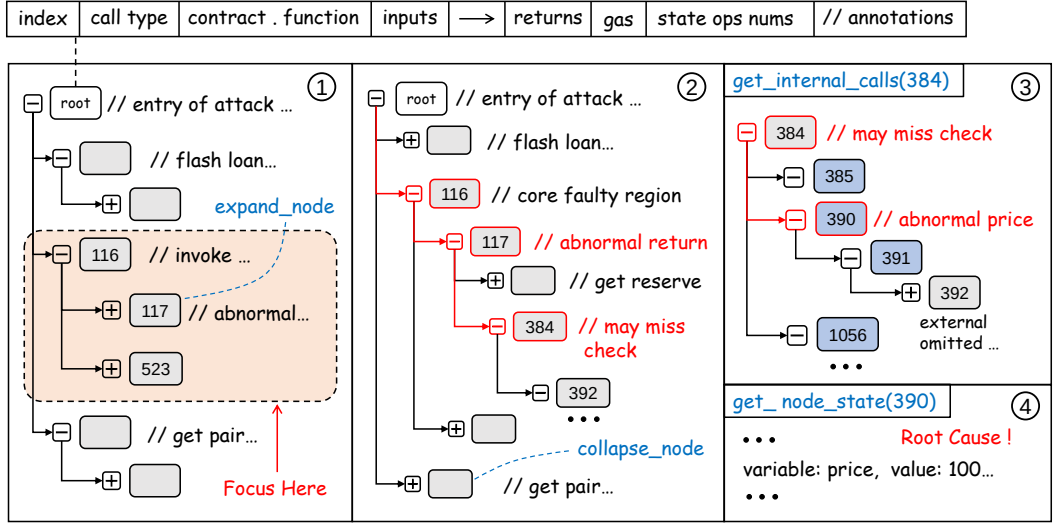


Fig. 6. Dynamic Trace Exploration via Semantics-driven Pruning

turn without conflicts. Once the transaction debugger deems the root cause identified, it calls the `ready_for_patch` tool to finish the analysis.

4.3.2 Dynamic Trace Exploration. To analyze the immense and noisy trace data effectively, TracePilot emulates the progressive debugging strategies of seasoned experts. Given an immense search space, inexperienced analysts suffer from the same attention deficits as LLMs. However, seasoned experts can focus their limited attention on critical fault points to extract attack logic. Experts use the external call trace as a skeleton, analyzing shallow calls to segment execution into functional phases. Experts prune irrelevant operations to focus on potential exploits, then selectively expand suspicious sub-trees. This iterative process is driven by evolving fault hypotheses. Moreover, TracePilot automates the expert workflow of using Debug mode in Phalcon [5] to examine the internal execution trace. Our approach begins with the external skeleton, then drills down into internal call traces of suspicious nodes for further analysis.

Interactive Simulator. We develop DebugSimulator, a trace management component that functions as a dashboard for agents. It maintains an external trace tree where each node encapsulates `view_expanded` and `annotation` attributes alongside trace data from Tenderly (details in Section 2.3). The transaction debugger can dynamically modify the visible trace tree using `expand_node` and `collapse_node` tools to toggle the `view_expanded` state. To prevent initial context overflow, we set the initial visibility depth to ≤ 2 , similar to Phalcon [5]. The render method generates a textual representation of all visible nodes. Each node is displayed in a structured format shown in Fig. 6. For unverified contracts, the simulator preserves function selectors. Fundamentally, this component acts as a semantic wrapper that filters noise while enabling agents to use external calls as pivots for investigating internal traces.

Semantics-Driven Exploration Strategy. TracePilot prunes the search space by extracting semantic features from trace nodes. As shown in Fig. 6, exploration begins with a trace tree limited to depth ≤ 2 to focus on top-level semantics. Guided by fault understanding and tasks, the transaction debugger employs the DebugSimulator to dynamically analyze suspicious function calls and prune irrelevant branches. Through exploration, the transaction debugger uses `update_annotations` tool to label nodes, thereby preserving annotations in DebugSimulator. These persisted annotations

enable the agent to segment functional phases and rapidly recall explored semantics, thereby focusing subsequent iterations on newly expanded sub-trees. Once concrete new insights are identified from the trace tree, the transaction debugger invokes `update_understanding` tool to refine global fault understanding and recalibrate the task tree. To navigate internal calls beyond the external trace tree, we developed the `get_internal_calls` tool (details in Section 4.3.4). This tool supports on-demand retrieval of internal traces for suspicious nodes, preventing context overflow while exposing critical internal execution details. Such granular visibility empowers TracePilot to efficiently identify root causes hidden deep within internal execution traces.

Context Length Management. To ensure reasoning continuity and prevent context overflow, TracePilot enforces a pruning threshold derived from the maximum window size. When the input exceeds this limit, the system triggers a mandatory cleanup: the agent is prompted to summarize the logic of low-value subtrees (e.g., redundant loops or irrelevant branches) before collapsing them. To maintain a persistent memory of closed branches, TracePilot records these summaries into the annotation field of the root node with a `[Collapsed]` prefix. This allows the agent to understand the pruned context without re-expanding the trace.

4.3.3 Cross-Transaction Analysis. Semantics-driven exploration reduces the search space, making complex cross-transaction analysis feasible. To optimize efficiency, TracePilot screens out the repetitive transactions common in the attack sequence. We leverage an LLM to deduplicate the attack transactions identified in Section 4.2.1, isolating representative samples for in-depth analysis.

In cross-transaction scenarios, the global understanding manager maintains the transaction hash under analysis. To facilitate attack analysis, we implement a `TraceSessionManager` for cross-transaction context switching. The `TraceSessionManager` maps the transaction hash to an isolated session containing a `DebugSimulator` and the trace data. This architecture guarantees data contextual isolation, providing a stable environment for reasoning across the attack sequence. The transaction debugger employs the `switch_transaction` tool to navigate to other transactions for analysis. This tool accepts three parameters: target transaction hash, interim fault insights with detailed evidence, and fix suggestions for current fault. `switch_transaction` tool requires the transaction debugger to distill key evidence and possible root causes into current insights before each transition. This ensures coherent cross-transaction reasoning and preserves key evidence from each transaction. Then, the global understanding manager archives the provided insights and fix suggestions into `analyzed_transactions`. By organizing these findings within `attack_stages`, the system constructs a cumulative attack logic chain. This structured history enables the transaction debugger to reference prior core operations and key evidence during subsequent analyses, thereby establishing cross-transaction state dependencies. To ensure diagnostic completeness, the system must traverse all unique attack transactions before generating the final report.

4.3.4 Debugging Toolkit. To support precise forensics, we implement a comprehensive toolkit based on the MCP [3]. This toolkit emulates the diagnostic environment of human experts, equipping the transaction debugger with granular evidence across source code, storage states, and internal execution traces. Table 1 summarizes the available tools and their specific functionalities.

Effective debugging requires flexible access to both static code and dynamic state. We provide source code tools at varying granularities, ranging from specific function snippets to full contract verification. This flexibility allows the debugger to adapt its focus based on the analysis result.

Critically, to capture the dynamic evolution of variables, we design the `get_node_state` tool. This tool aggregates a unified execution snapshot for any target node, encompassing input/output arguments, variable states before and after the call, storage slot accesses, and log events. These features enable the transaction debugger to assess function health and track variable transitions within a single invocation. Specifically, granular storage records reveal state evolution, allowing

Table 1. Overview of the Debugging Toolkit.

Category	Tool Name	Description & Functionality
Source Code	get_contract_source_code	Retrieves the full source code of the contract.
	get_function_source_code	Returns the specific function source combined with the call-site snippet for immediate context.
	analyze_unverified_contract	Resolves unverified bytecode via Sourcify API [24] (signatures) and Heimdall [4] (decompilation).
Runtime Features	get_internal_calls	Return internal call trace tree of a specific node.
	get_node_state	Aggregates a full execution snapshot: args, return values, variable changes, storage slot accesses, and logs.
Workflow Control	update_understanding	Synchronizes new insights into global fault understanding.
	switch_transaction	Switches the analysis context to a different transaction.
	ready_for_patch	Concludes the analysis phase and submits a JSON-formatted root cause report and fix suggestions.

the agent to detect anomalies. With this tool, our system can also identify faulty functions when they do not appear in the execution trace, such as constructor configuration errors.

To diagnose faults within internal function calls, we implemented the `get_internal_calls` tool. Inspired by the Debug mode of Phalcon [5], this tool extends the visibility of TracePilot beyond the external call trace into granular internal logic. Upon invocation, the tool performs a depth-first search (DFS) from a target node to construct a internal call trace tree. As shown in Fig. 6, this traversal captures all internal sub-calls but halts immediately upon encountering the next layer of external calls. These external nodes are retained as boundaries while their subsequent execution is omitted, allowing the agent to atomically inspect them via separate invocations if necessary. Furthermore, since internal nodes inherently possess unique indexes, revealing this structure significantly extends the analysis scope. It enables transaction debuggers to leverage these now-visible indexes to apply forensic tools to deeply nested logic.

Finally, the toolkit includes control tools (e.g., `ready_for_patch`) that allow transaction debugger to manage the analysis lifecycle, synchronize insights, and trigger the subsequent verification phase.

4.4 Patch Verification

To bridge the gap between diagnostic outputs and actionable trust, TracePilot introduces an automated, closed-loop patch verification mechanism. This phase validates the root cause analysis by synthesizing and testing contract patches in the simulation environment.

We orchestrate this workflow using two specialized agents: the code patcher and the attack judge. As shown in Fig. 7, the process initiates by consuming the fix suggestions in JSON format generated by the transaction debugger. The `faulty_functions` field lists identified faulty points, where each entry contains a fix strategy, core patch snippets, and an unique identifier tuple of transaction hash and node index. Our work leverages this tuple to identify the specific node within execution trace trees, thereby retrieving comprehensive metadata like contract source code and compilation artifacts. Subsequently, we construct a mapping for all target contract source codes, linking the tuple of contract address and filename to the corresponding source code.

The code patcher converts abstract suggestions into executable patches. It employs Aider-style diff blocks [2] to ensure conflict-free application. Each block targets a specific contract (by address and filename) and defines changes via SEARCH/REPLACE segments. TracePilot applies these

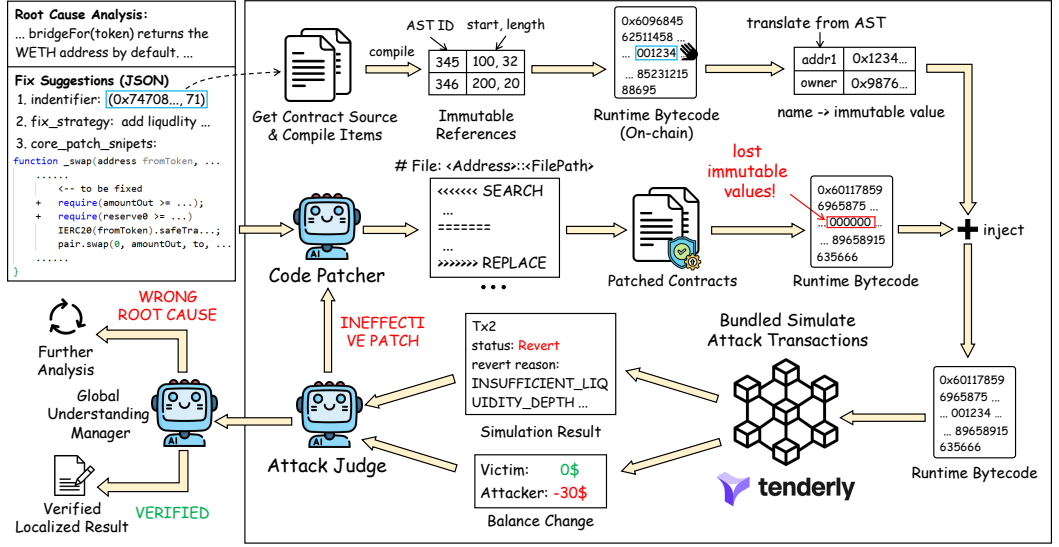


Fig. 7. Stage 3: Automated Patch Generation and Verification.

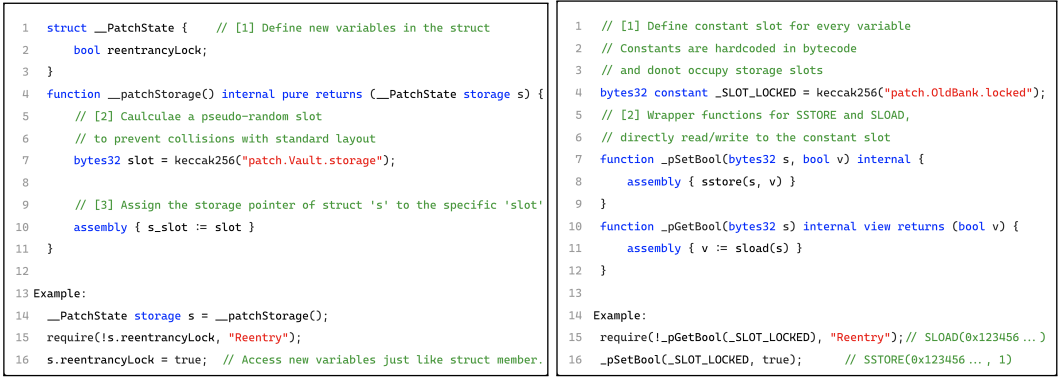
(a) Solidity $\geq 0.6.0$ (b) Solidity $< 0.6.0$

Fig. 8. Version-adaptive Strategies for Resolving Storage Slot Collisions.

patches using regular expressions, stripping comments to prevent matching failures caused by LLM hallucinations. After application, the patched contracts are recompiled to generate runtime bytecode. To validate the remediation, we invoke the Tenderly bundled transactions simulation API to replay the attack transactions. According to the documentation [29], the bundled simulation API executes transactions sequentially, applying state changes cumulatively to subsequent transactions. If any transaction fails, the execution of subsequent transactions is halted. Crucially, we utilize the state_objects dictionary in the API request to override the on-chain runtime bytecode of faulty contracts with our patched versions. This mechanism achieves an in-place upgrade within the simulation environment.

Directly substituting runtime bytecode introduces two critical technical challenges: storage slot collisions and the bypassing of constructors.

Storage Slot Collision. Patching often requires introducing new state variables, such as reentrancy guards. Directly declaring these variables in the source code shifts the storage slots of existing variables. This leads to collisions where original data maps to incorrect slots. We resolve

this problem by mapping new variables introduced by patches to pseudo-random storage slots. This logically isolates the patch state from the original storage layout. As illustrated in Fig. 8, we employ a version-adaptive strategy to handle syntax differences.

State Reconstruction. Since our approach bypasses the standard deployment phase, original constructors are precluded from execution. To ensure the remediated contract correctly inherits its existing states, we devise reconstruction mechanisms for both storage states and immutable variables.

1) *Lazy Initialization:* To modify the initialization logic without running the constructor, the code patcher employs a lazy initialization pattern. It synthesizes an initialization guard function, which executes assignment and state-reset logic only upon its first invocation. By mandating that all public entry functions invoke this guard prior to business logic, TracePilot ensures a seamless state transition and logical upgrade without contract redeployment.

2) *Immutable Variable Transplantation:* Immutable variables are hardcoded into the runtime bytecode upon deployment and assigned only within the constructor. Consequently, recompiling the patched contract resets all immutable variables to zero, compromising logical integrity. To address this challenge, we adopt an immutable variable transplantation strategy as shown in Fig. 7. The system parses the AST and immutableReferences of both original and patched contracts to map variable names to their exact bytecode offsets. Then, the transplanter extracts the solidified variable values from the original on-chain bytecode and injects them into the new bytecode at the identified offsets. This ensures the patched contract inherits critical environmental parameters despite bypassing the construction phase.

Post-simulation, the attack judge evaluates remediation efficacy via multi-dimensional analysis. It first filters revert reasons to distinguish valid interceptions from execution errors like Out-Of-Gas. Next, the agent verifies balance changes; if the attack completes without profit extraction, the defense is deemed effective. For successful mitigations, the agent reviews the patch code to ensure the patch does not disrupt legitimate business logic. Only when the attack is blocked and the patch quality is verified does the system finalize the fault root cause report. In cases of defense failure, the system analyzes the execution trace and reviews the patch code. If the patch contains implementation bugs, the code patcher is instructed to refine the logic. However, if the patch is correct but fails to block the attack, the root cause hypothesis is considered incorrect, triggering a rollback to micro trace analysis.

5 Evaluations

To evaluate TracePilot, we conduct experiments to address the following research questions (RQs):

RQ1: Comparative experiment. How does TracePilot perform compared to SOTA techniques?

RQ2: Performance on Patch Verification. How does proposed patch verification perform?

RQ3: Ablation study. Does each component contribute to the overall performance of TracePilot?

RQ4: Efficiency and Cost Analysis. How efficient and cost-effective is TracePilot?

5.1 Experimental Settings

5.1.1 Implementation Details. We developed the system using Python, comprising over 11k lines of code across 87 files. For the underlying foundation model, we utilize DeepSeek-V3.2 [19] for all inference and experimental tasks. This model meets the real-world demands of DApp fault diagnosis by offering high-performance reasoning at a significantly lower computational cost. Maintaining a sole model throughout the experiments, we ensure that the evaluation focuses on the validity of the framework design rather than the capabilities of the underlying model.

To balance thoroughness with efficiency, we establish strict termination criteria for the iterative process. The analysis stage proceeds for up to 50 turns, or concludes early if 15 turns yield no new

Table 2. Results of comparative studies with state-of-the-art. $Top - N(\%) = |Top - N|/\#faults$. $R@1 \uparrow$ and $P \uparrow$ represent the improvement of TracePilot compared to other methods in Top-1 Recall and Precision.

Model	Recall@Top-N (%)			Precision	R@1 $\uparrow$	P $\uparrow$
	N=1	N=3	N=5			
DAppFL	12.75	19.46	22.82	4.56%	458.0%	1605.7%
FaultSeeker	31.08	49.32	50.67	23.04%	128.9%	237.6%
TracePilot	71.14	71.14	71.14	77.78%	-	-

insights. The system then advances to patching. If patching (3 turns) or bug-fixing (15 turns) fails to converge, the workflow backtracks to the analysis stage to refine the hypothesis. The entire process is strictly limited to 3 global cycles or 3 hours. Reaching either limit triggers termination and the generation of a final report carrying a prominent warning regarding its unverified status.

5.1.2 Data Collection. Our dataset comprises 149 DApp fault cases reported before January 2026. The construction process began with 115 single-transaction cases collected from FaultSeeker. We conducted a comprehensive manual review of these cases by cross-referencing them with public security reports from authoritative organizations, e.g., DeFiHackLabs [26] and the Phalcon security incidents library [6]. To enhance the diversity of our dataset, we further incorporated 23 new cross-transaction attacks and 11 single-transaction attacks. The final dataset categorizes into 28 cross-transaction and 121 single-transaction attacks. For each case, we retrieved the security reports and audited their contents to extract the definitive root-cause functions and detailed analysis report.

5.1.3 Comparative Methods. To answer RQ1, we evaluate TracePilot against two SOTA baselines: FaultSeeker and DAppFL. Specifically, our literature review highlights FaultSeeker as the representative for LLM-based approaches, while DAppFL leads the domain of learning-based methods. Moreover, general-purpose fault localization methods cannot be directly applied to the DApp faults [35, 38], as they lack both specialized domain knowledge and the required key data sources, such as detailed error tracebacks. As FaultSeeker is limited to processing single attack transactions, we retained the pivotal attack transaction from the dataset for evaluation.

5.1.4 Measurement. We use Recall at top-1, top-3, and top-5 ranking positions to assess the retrieval capability, and Precision to quantify the ratio of correctly localized faults to the total number of suspicious functions.

5.2 Comparative Experiment (RQ1)

To answer RQ1, we compare TracePilot with state-of-the-art fault localization techniques on our dataset. Table 2 presents the detailed experimental results. Following recent evidence that LLMs align closely with human expert judgment, we employ an LLM-based auditor for evaluation.

Unlike existing tools that output ranked lists of suspicious functions, TracePilot generates descriptive diagnostic reports. Since advanced LLMs have demonstrated strong alignment with human judgment in evaluation tasks [7, 42], we employ an LLM-based evaluator to verify the results. Specifically, the evaluator compares the generated report against the expert-verified ground truth. A localization is considered successful if the generated report explicitly identifies the ground-truth vulnerable function as the root cause of the fault.

As shown in Table 2, TracePilot significantly outperforms all baseline methods across all metrics. Specifically, TracePilot achieves a **Top-1 Recall of 71.14%**, which is a **128.9%** improvement over the strongest baseline, FaultSeeker (31.08%), and more than **5.5** times to the performance of DAppFL (12.75%). In terms of **Precision**, TracePilot reaches **77.78%**, outperforming FaultSeeker (23.04%) and

Table 3. Localization results vs. patch verification.

Result	Patch Pass	Patch Fail	Total
Hit	84 (TP)	22 (FN)	106
Miss	24 (FP)	19 (TN)	43
Total	108	41	149

Table 4. Key reliability metrics.

Metric	Count	Rate
Executability	124/149	83.2%
Fix Success	108/149	72.5%
Consistency	84/106	79.3%
Reliability	84/108	77.8%

DAppFL (4.56%) by **237.6%** and **1605.7%**, respectively. These results demonstrate that TracePilot is not only more capable of finding the true root cause but also provides much more reliable results.

Unlike baselines that rely on ranked lists with high-recall and low-precision, TracePilot provides a singular pinpoint diagnosis. This precision stems from its recursive cognitive loop. By constructing a complete attack logic chain from trace evidence, TracePilot logically identifies the unique root cause instead of outputting multiple noisy candidates. For consistency in Table 2, we report the same Top-1 value for its Top-3 and Top-5 metrics.

Answer of RQ1: TracePilot outperforms SOTA methods with a **71.14%** Top-1 Recall and **77.78%** Precision, delivering precise pinpoint diagnoses rather than noisy suspicion lists.

5.3 Performance on Patch Verification (RQ2)

To evaluate the reliability of self-verification, we analyze its performance on 149 cases from three perspectives: executability, fix effectiveness, and localization-fix consistency. As shown in Table 3, TracePilot achieves an Executability Rate of 83.22%, with 124 out of 149 patches successfully passing compilation. This indicates that the code patcher can generate syntactically correct and deployable patches even for legacy or complex contracts. More importantly, the Fix Success Rate reaches 72.48% (108/149), where the attack judge confirms the successful interception of the original exploit in the simulation testnet. To further scrutinize the reliability of using patches as evidence for localization, we examine the Localization-Fix Consistency. Among the 106 cases where the root cause was correctly localized, the patch successfully passed verification in 84 cases, yielding a consistency of 79.25%. This high consistency demonstrates that the fault understanding of TracePilot is not merely superficial but can be translated into effective defensive logic. However, we observe a Fix Attrition (False Negative) rate of 20.75% (22/106), where the root cause was correctly identified but the patch failed to pass. We discuss this false negative cases in Section 6.1. We investigated the False Fix (False Positive) phenomenon, where an incorrect localization still yields a passing patch. This occurred in 22.22% (24/108) of successful fixes, typically through brute-force solutions, such as adding a global reentrancyGuard that intercepts a specific attack path without addressing the root cause. The attack judge effectively contains this rate by filtering out patches that compromise business logic. In the remaining 77.78% of cases, localization and patching aligned perfectly, confirming that a passing patch is a high-fidelity indicator of correct fault localization.

Answer of RQ2: The self-verification is reliable, achieving **83.2%** patch executability and **72.5%** fix success. The **79.3%** consistency confirms that a successful patch leads to correct fault localization result.

5.4 Ablation Study (RQ3)

To evaluate the contribution of each component, we compared TracePilot against three variants: (1) w/o Macro Transaction Analysis, omitting high-level anomaly reports; (2) w/o Dynamic Trace

Table 5. Ablation study on key components.

Configuration	Top-1@Recall	Precision
Full TracePilot	71.14%	77.78%
w/o Macro Analysis	56.38%	65.12%
w/o Trace Exploration	43.62%	50.00%
w/o Verification	54.36%	64.80%

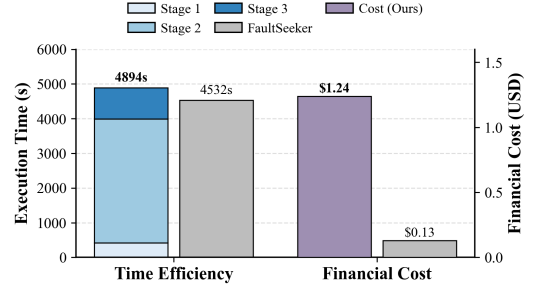


Fig. 9. Efficiency and cost evaluation.

Exploration, replacing the recursive loop with a single-pass linear analysis; and (3) w/o Patch Verification, removing the feedback loop to prevent backtracking and self-correction (Table 5).

Results show the full framework significantly outperforms all ablated versions. The most critical component is Dynamic Exploration. Its removal causes the sharpest drop in Top-1 Recall (to 43.62%) and Precision (to 0.500), proving that simple one-shot analysis fails against the noise of massive execution traces and that iterative reasoning is essential. Removing Macro Transaction Analysis also degrades performance (Recall 56.38%), as the agent lacks the global context of asset flows to guide its search, often misidentifying root causes in complex DApp faults. Finally, the absence of Patch Verification lowers Recall to 54.36%. This confirms that the closed-loop feedback mechanism is vital for correcting hypotheses, ensuring that diagnoses are backed by concrete execution evidence rather than just theoretical assumptions.

Answer of RQ3: All components contribute to fault localization effectiveness, with dynamic trace exploration serving as the foundational enabler.

5.5 Efficiency and Cost Analysis (RQ4)

In RQ5, we evaluate the runningtime and financial costs of TracePilot when using the DeepSeek-V3.2 API. As both are multi-agent frameworks, Fig. 9 shows the average execution time across different stages and the financial costs compared to the state-of-the-art tool, FaultSeeker.

TracePilot completes the end-to-end localization and verification process in an average of 4,893.81 seconds (≈ 1.36 hours). Despite handling complex cross-transaction scenarios, its total runtime remains comparable to FaultSeeker (4,531.50 seconds). While Stage 3 (Verification) introduces an additional overhead of 904.34 seconds, this stage is critical for automating the validation process, which typically requires hours of manual trial-and-error by human experts. Given that human experts require an average of 16.7 hours for manual localization, TracePilot completes the end-to-end process in just 1.36 hours, highlighting its potential for rapid security response.

We estimate the financial cost based on the input (\$0.28 per 10^6 tokens) and output (\$0.42 per 10^6 tokens) prices of the DeepSeek-V3.2 API [8]. The average cost per incident for TracePilot is \$1.24. Although this is higher than FaultSeeker, the cost remains negligible compared to the potential financial losses in blockchain exploits or the high labor costs of security experts. The marginal increase in expenditure is a justifiable trade-off for the enhanced diagnostic accuracy and automated verification capabilities provided by TracePilot. The additional cost is a reasonable trade-off for the superior diagnostic capabilities provided by TracePilot.

Answer of RQ4: TracePilot costs only **\$1.24** per case and finishes in **1.36 hours**. It matches the speed of non-verifiable baselines while reducing manual analysis time by **91.8%** compared to human experts.

6 Discussion

6.1 Qualitative Analysis of Failures

To rigorously assess the limitations of TracePilot, we conducted a qualitative analysis of cases where root cause localization was inaccurate or incomplete. Despite superior overall performance, two primary failure categories emerged.

The first limitation lies in the failure to identify the precise root cause of extremely complex fault, such as the *Audius* governance exploit. The root cause of this exploit is a storage slot collision in the initialize function. While TracePilot correctly identified the faulty function, our approach fails to determine exactly how the attacker triggered the vulnerability. We attribute this to the lack of deep domain expertise. Instead of analyzing intricate logic, LLM biases its analysis toward common issues like missing permission checks. However, even in this failure, TracePilot significantly reduced manual auditing effort by narrowing the search space.

The second category concerns partial remediation, exemplified by the *Alchemix* incident. The system marked verification as successful because the generated patch effectively blocked the attack; however, the patch only fixed one of the trigger conditions, missing the true root cause. This highlights a limitation in the Patch-based verification strategy when handling exploits with multiple failure conditions. Nevertheless, the ability to automatically generate a functional defense remains critical for emergency response, offering a reference for immediate protection while experts perform exhaustive analysis.

6.2 Threats to Validity

In this section, we discuss the potential threats to the validity of our study and the measures taken to mitigate them.

Internal Validity. The primary internal threat is the stochastic nature of LLMs. Since TracePilot relies on LLMs for reasoning, the model may generate different analysis paths or conclusions for the same input, potentially leading to inconsistent localization results. To mitigate this, we implemented two key strategies. First, we strictly control the generation process by fixing the temperature parameter and setting specific turn caps. Second, we introduced the automated patch verification module. By replaying the attack process in a simulated environment, TracePilot verifies the LLM's reasoning against actual execution results, effectively minimizing the impact of hallucinations on the final diagnosis.

External Validity. External threats mainly concern the dataset representation. For cross-transaction attacks, a critical challenge is whether the dataset accurately preserves all key state dependencies across transactions. To ensure data integrity, we extracted expert-labeled attack sequences from official reports on authoritative security platforms. We then conducted manual audits of the transaction history surrounding these events to analyze attacker behaviors and ensure that all critical state changes exploited during the incident were fully preserved in our dataset.

7 Related Work

Fault localization techniques aim to identify the program elements likely responsible for faults. And classical fault localization methods can be broadly categorized into four types: spectrum-based, mutation-based, learning-based, and language model-based techniques.

Classical fault localization. Spectrum-based techniques [1, 13, 17] utilize coverage information extracted from both passed cases and failed cases to assess the suspiciousness of specific code

snippets. Mutation-based techniques [12, 21, 22] modify the source code of a faulty program (e.g., $x = x + y$ to $x = x * y$), create versions known as mutants, and analyze the outcomes across these mutants to pinpoint faults. Learning-based techniques [16, 18, 20, 35, 41] model coverage data, code metrics, and error messages as features. Then, the features are fed into machine learning models to produce a ranked list of suspicious program elements. Recently, language models have also been used in fault localization tasks [14, 34, 37]. The language models can directly analyze multi-dimensional data, including source code, error messages, and execution traces.

However, despite the success of the classical fault localization techniques in traditional software, these techniques face challenges when applied to DApps. Existing research [35, 38] indicates that classical fault localization methods fail to locate the root cause of DApp faults. For one thing, the classical fault localization methods cannot analyze cryptocurrency flow, which is the distinctive feature of DApp faults. For another, DApp faults often lack error messages, which is a critical feature relied upon by classical techniques. Consequently, traditional fault localization techniques are ill-suited for direct application to DApp fault localization tasks.

DApp fault localization. Note that some studies have attempted to address the specific challenges of DApp fault localization. For instance, DAppFL [35] combines cryptocurrency flow analysis with graph neural networks to rank DApp functions based on their suspiciousness. Malo [38] employs graph neural networks to analyze the dynamic control flow graphs, aiming to identify the basic blocks likely responsible for faults. FaultSeeker [25] leverages large language models to match suspicious functions within execution traces.

However, current DApp fault localization methods also suffer from significant limitations. First, constrained by GNNs, the current DApp fault localization methods rely on training data, which is often scarce in the DApp fault localization task. Second, the current methods depend on expert-defined patterns rather than proactively exploring massive cross-transaction DApp fault data, leading to false negatives and false positives. Finally, the generated explanations lack automated verification. Therefore, the experts have to put in additional effort to validate the correctness of fault explanations.

In contrast to existing fault localization techniques, the proposed approach offers the following distinct advantages: 1) It does not require training data; 2) It mimics the debugging reasoning of human experts without relying on predefined matching patterns, thereby reducing false negatives; and 3) It validates the explanation results through PoC generation, ensuring the correctness of fault explanations.

8 Conclusion

This paper presents TracePilot, the *first* self-verifiable framework for automated DApp fault localization across transactions. By internalizing the expert cognitive process into a systematic workflow, TracePilot effectively isolates intricate, cross-transaction fault logic, and provide verifiable evidence through its automated patch verification mechanism. Experimental results on 149 real-world cases demonstrate that TracePilot achieves 71.14% Top-1 recall, outperforming state-of-the-art methods by 128.9%. The results substantiate that TracePilot offers an effective and reliable solution for automated, post-incident analysis in the Web3 ecosystem, enhancing both diagnostic accuracy and operational trustworthiness.

9 Data Availability

To facilitate replication and future research, we make our source code, extended dataset, and experimental details publicly available at the repository: <https://anonymous.4open.science/r/TracePilot-F8A6/>.

References

- [1] Rui Abreu, Peter Zoetewij, and Arjan J.c. Van Gemund. 2006. An Evaluation of Similarity Coefficients for Software Fault Localization. In *2006 12th Pacific Rim International Symposium on Dependable Computing (PRDC'06)*. 39–46. doi:10.1109/PRDC.2006.18
- [2] Aider-AI. 2026. Aider - AI Pair Programming in Your Terminal. <https://github.com/Aider-AI/aider>.
- [3] Anthropic. 2026. Model Context Protocol. <https://github.com/modelcontextprotocol>.
- [4] Jonathan Becker. 2024. Heimdall - an advanced EVM toolkit. <https://github.com/Jon-Becker/heimdall-rs>.
- [5] BlockSec. 2026. Dive into transaction to act wisely- blocksec phalcon explorer. <https://blocksec.com/phalcon/explorer>.
- [6] BlockSec. 2026. Security Incidents Library. <https://blocksec.com/security-incident>.
- [7] Cheng-Han Chiang and Hung-yi Lee. 2023. Can Large Language Models Be an Alternative to Human Evaluations?. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 15607–15631. doi:10.18653/v1/2023.acl-long.870
- [8] DeepSeek. 2026. Models & Pricing. https://api-docs.deepseek.com/quick_start/pricing/.
- [9] DeFiHackLabs. 2022. OnChain Transaction Debugging: 3. Write Your Own PoC (Price Oracle Manipulation). https://github.com/SunWeb3Sec/DeFiHackLabs/tree/main/academy/onchain_debug/03_write_your_own_poc/en
- [10] DeFiLlama. 2026. DeFiLlama API Reference: Get historical prices of tokens by contract address. <https://api-docs.defillama.com/#tag/coins/get/prices/historical/{timestamp}/{coins}>.
- [11] DeFiLlama. 2026. Total Value Locked in DeFi. <https://defillama.com/> Accessed: 2026-01-28.
- [12] Joran J. Honig, Maarten H. Everts, and Marieke Huisman. 2019. Practical Mutation Testing for Smart Contracts. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology*. Springer International Publishing, Cham, 289–303.
- [13] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering* (Long Beach, CA, USA). 273–282. doi:10.1145/1101908.1101949
- [14] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. *Proc. ACM Softw. Eng.* 1, FSE, Article 64 (jul 2024), 23 pages. doi:10.1145/3660771
- [15] Chenliang Li, He Chen, Ming Yan, Weizhou Shen, Haiyang Xu, Zhikai Wu, Zhicheng Zhang, Wenmeng Zhou, Yingda Chen, Chen Cheng, Hongzhu Shi, Ji Zhang, Fei Huang, and Jingren Zhou. 2023. ModelScope-Agent: Building Your Customizable Agent System with Open-source Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Yansong Feng and Els Lefever (Eds.). Association for Computational Linguistics, Singapore, 566–578. doi:10.18653/v1/2023.emnlp-demo.51
- [16] Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. 2019. DeepFL: integrating multiple fault diagnosis dimensions for deep fault localization. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Beijing, China) (ISSTA 2019). 169–180. doi:10.1145/3293882.3330574
- [17] Xia Li and Lingming Zhang. 2017. Transforming programs and tests in tandem for fault localization. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 92 (oct 2017), 30 pages. doi:10.1145/3133916
- [18] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2021. Fault Localization with Code Coverage Representation Learning. In *Proceedings of the 43rd International Conference on Software Engineering* (Madrid, Spain) (ICSE '21). IEEE Press, 661–673. doi:10.1109/ICSE43902.2021.00067
- [19] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. 2025. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556* (2025).
- [20] Yiling Lou, Qihao Zhu, Jinhao Dong, Xia Li, Zeyu Sun, Dan Hao, Lu Zhang, and Lingming Zhang. 2021. Boosting coverage-based fault localization via graph-based representation learning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) (ESEC/FSE 2021). 664–676. doi:10.1145/3468264.3468580
- [21] Seokhyeon Moon, Yunho Kim, Moonzoo Kim, and Shin Yoo. 2014. Ask the Mutants: Mutating Faulty Programs for Fault Localization. In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. 153–162. doi:10.1109/ICST.2014.28
- [22] Mike Papadakis and Yves Le Traon. 2015. Metallaxis-FL: mutation-based fault localization. *Softw. Test. Verif. Reliab.* 25, 5–7 (aug 2015), 605–628. doi:10.1002/stvr.1509
- [23] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2025. SoapFL: A Standard Operating Procedure for LLM-Based Method-Level Fault Localization. *IEEE Transactions on Software Engineering* 51, 4 (2025), 1173–1187. doi:10.1109/TSE.2025.3543187
- [24] Sourcify. 2026. Sourcify signature service API. <https://docs.sourcify.dev/docs/api/>.

- [25] Kairan Sun, Zhengzi Xu, Kaixuan Li, Lyuye Zhang, Yuqiang Sun, Liwei Tan, and Yang Liu. 2025. FAULTSEEKER: LLM-Empowered Framework for Blockchain Transaction Fault Localization. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering, ASE*. IEEE/ACM, Seoul, South Korea.
- [26] SunWeb3Sec. 2026. DeFiHackLabs. <https://github.com/SunWeb3Sec/DeFiHackLabs>.
- [27] XAgent Team. 2023. XAgent: An Autonomous Agent for Complex Task Solving. <https://github.com/OpenBMB/XAgent>.
- [28] Tenderly. 2026. Tenderly Explorer. <https://tenderly.co/developer-explorer>.
- [29] Tenderly. 2026. Tenderly Simulate bundled transactions API document. <https://docs.tenderly.co/reference/api#/operations/simulateTransactionBundle>.
- [30] Tenderly. 2026. Tenderly Simulate transaction API document. <https://docs.tenderly.co/reference/api#/operations/simulateTransaction>.
- [31] Jianing Wang, Junda Wu, Yupeng Hou, Yao Liu, Ming Gao, and Julian McAuley. 2024. InstructGraph: Boosting Large Language Models via Graph-centric Instruction Tuning and Preference Alignment. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 13492–13510. doi:10.18653/v1/2024.findings-acl.801
- [32] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf
- [33] Gavin Wood et al. 2014. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper* 151, 2014 (2014), 1–32.
- [34] Yonghao Wu, Zheng Li, Jie M. Zhang, Mike Papadakis, Mark Harman, and Yong Liu. 2023. Large Language Models in Fault Localisation. arXiv:2308.15276 [cs.SE] <https://arxiv.org/abs/2308.15276>
- [35] Zhiying Wu, Jiajing Wu, Hui Zhang, Ziwei Li, Jiachi Chen, Zibin Zheng, Qing Xia, Gang Fan, and Yi Zhen. 2024. DAppFL: Just-in-Time Fault Localization for Decentralized Applications in Web3. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA*. ACM, Vienna, Austria, 137–148. doi:10.1145/3650212.3652116
- [36] Zhiying Wu, Jiajing Wu, Hui Zhang, Zibin Zheng, and Weiqiang Wang. 2025. Hunting in the Dark Forest: A Pre-trained Model for On-chain Attack Transaction Detection in Web3. In *Proceedings of the ACM on Web Conference 2025*. 4519–4530.
- [37] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent Hellendoorn. 2024. Large Language Models for Test-Free Fault Localization. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Article 17, 12 pages. doi:10.1145/3597503.3623342
- [38] Hui Zhang, Jiajing Wu, Zhiying Wu, Zhe Chen, Dan Lin, Jiachi Chen, Yuren Zhou, and Zibin Zheng. 2025. Malo in the Code Jungle: Explainable Fault Localization for Decentralized Applications. *IEEE Transactions on Software Engineering* 51, 7 (2025), 2197–2210. doi:10.1109/TSE.2025.3578816
- [39] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Xiaosong Huang, Chiming Duan, and Ying Li. 2025. AgentFM: Role-Aware Failure Management for Distributed Databases with LLM-Driven Multi-Agents. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (Clarion Hotel Trondheim, Trondheim, Norway) (FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 525–529. doi:10.1145/3696630.3728492
- [40] Wei Zhang, Hongcheng Guo, Jian Yang, Zhoujin Tian, Yi Zhang, Yan Chaoran, Zhoujun Li, Tongliang Li, Xu Shi, Liangfan Zheng, and Bo Zhang. 2024. mABC: Multi-Agent Blockchain-inspired Collaboration for Root Cause Analysis in Micro-Services Architecture. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Miami, Florida, USA, 4017–4033. doi:10.18653/v1/2024.findings-emnlp.232
- [41] Zhuo Zhang, Yan Lei, Xiaoguang Mao, and Panpan Li. 2019. CNN-FL: An Effective Approach for Localizing Faults using Convolutional Neural Networks. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 445–455. doi:10.1109/SANER.2019.8668002
- [42] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 46595–46623. https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf
- [43] Jialong Zhou, Lichao Wang, and Xiao Yang. 2025. GUARDIAN: Safeguarding LLM Multi-Agent Collaborations with Temporal Graph Modeling. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=6j9xJpBjm>

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009